

```

import asyncio
import aiohttp
from aiogram import Bot, Dispatcher, types
from aiogram.filters import Command,
CommandObject
from aiohttp import web
import time
from collections import deque
import os

# ===== НАСТРОЙКИ
=====
# ВАЖНО: токен ТОЛЬКО из переменной
окружения. Никогда не хардкодь его в файле,
# иначе при пуше на GitHub он утечёт даже
из приватного репозитория (кэши, форки,
# коллабораторы). На Render: Settings ->
Environment -> добавь BOT_TOKEN.
BOT_TOKEN = os.environ.get("BOT_TOKEN")
if not BOT_TOKEN:
    raise RuntimeError("Не задана
переменная окружения BOT_TOKEN")

settings = {
    # --- Цена ---
    "percent": 3.0,          # Мин. % РОСТА
    # --- Цены в окне (триггер пампа) ---
    "window_min": 15,       # Окно анализа
    # --- для "percent" (мин) — по умолчанию 15 мин ---
    "check_interval": 30,   # Как часто

```

```
проверять (сек)
    "min_volume": 100000,    # Мин. объём
24ч ($) — отсекаем неликвид

    # Пороги роста (floor = минимум, чтобы
алерт вообще сработал)
    "day_min_rise": 0.0,     # Мин. % роста
за 24ч (0 = выкл)
    "week_min_rise": 0.0,    # Мин. % роста
за 7 дней (0 = выкл)
    "month_min_rise": 0.0,   # Мин. % роста
за 30 дней (0 = выкл)

    # Потолки роста (ceiling = максимум,
чтобы отсечь уже "перегретые" монеты)
    "day_max_rise": 0.0,     # Скрыть, если
рост за 24ч больше X% (0 = выкл)
    "week_max_rise": 0.0,    # Скрыть, если
рост за 7д больше X% (0 = выкл)
    "month_max_rise": 0.0,   # Скрыть, если
рост за 30д больше X% (0 = выкл)

    # --- Объём (новое) ---
    # % превышения объёма текущего
(последнего закрытого) часа над средним
    # объёмом ЭТОГО ЖЕ часа суток за
последние 7 дней (честное сравнение).
    "vh_percent": 0.0,       # 0 = выкл

    # % превышения объёма последней
закрытой минуты над средним объёмом
    # минутных свечей за последние ~24ч
(упрощение вместо "той же минуты неделю
    # назад" — см. пояснение в чате).
```

```

        "vm_percent": 0.0,          # 0 = выкл

        "cooldown_min": 5,         # Мин. пауза
от повторного алерта по той же монете
        "chat_id": None,
        "channel_id": None
    }

    price_history = {}              # symbol ->
deque[(ts, price)]
    blacklist = set()
    alert_memory = {}              # symbol ->
{"time": first_alert_ts, "last_msg": ts,
 "price": price}

# Кэши бейзлайнов объёма, чтобы не долбить
klines-эндпоинт на каждой итерации
BASELINE_TTL = 900                # обновлять
раз в 15 минут на пару
    hour_baseline_cache = {}       # symbol ->
{"ts": fetched_at, "avg": float, "cur":
 float}
    day_baseline_cache = {}       # symbol ->
{"ts": fetched_at, "avg": float, "cur":
 float}

    bot = Bot(token=BOT_TOKEN)
    dp = Dispatcher()

    def fmt_pct(key):
        val = settings[key]
        return "Выкл" if val == 0 else f"
{val}%"

```

```

# ===== TELEGRAM UI
=====

@dp.message(Command("start"))
async def start_cmd(message:
types.Message):
    settings["chat_id"] = message.chat.id
    await message.answer(
        "🚀 <b>Pump/Volume-сканер MEXC
запущен</b>\n"
        "Ниже — каждая команда, что она
делает, пример и <b>текущее
значение</b>.\n\n"

        "⚙️ <b>ЦЕНА</b>\n"
        f"/p 5 — мин. % роста цены в окне
(сам триггер алерта)\n"
        f"    L сейчас: <b>
{settings['percent']}%</b>\n"
        f"/t 15 — размер окна для /p, в
минутах\n"
        f"    L сейчас: <b>
{settings['window_min']} мин</b>\n"
        f"/d 5 — мин. % роста за 24ч, чтобы
алерт сработал (0=выкл)\n"
        f"    L сейчас: <b>
{fmt_pct('day_min_rise')}</b>\n"
        f"/dmax 20 — потолок роста за 24ч:
скрыть, если рост больше (0=выкл)\n"
        f"    L сейчас: <b>
{fmt_pct('day_max_rise')}</b>\n"
        f"/wmin 10 — мин. % роста за 7 дней

```

```

(0=выкл)\n"
    f"    L сейчас: <b>
{fmt_pct('week_min_rise')}</b>\n"
    f"/w 50 – потолок роста за 7 дней
(0=выкл)\n"
    f"    L сейчас: <b>
{fmt_pct('week_max_rise')}</b>\n"
    f"/mmin 20 – мин. % роста за 30
дней (0=выкл)\n"
    f"    L сейчас: <b>
{fmt_pct('month_min_rise')}</b>\n"
    f"/m 100 – потолок роста за 30 дней
(0=выкл)\n"
    f"    L сейчас: <b>
{fmt_pct('month_max_rise')}</b>\n\n"

    "⚙ <b>ОБЪЁМ</b>\n"
    f"/vh 200 – мин. % превышения
объёма текущего часа над средним по этому
же часу суток за последние ~7 дней
(0=выкл)\n"
    f"    L сейчас: <b>
{fmt_pct('vh_percent')}</b>\n"
    f"/vm 300 – мин. % превышения
объёма текущей минуты над средним объёмом
минут за последние сутки (0=выкл)\n"
    f"    L сейчас: <b>
{fmt_pct('vm_percent')}</b>\n"
    f"/v 200000 – мин. объём торгов за
24ч в $, ниже которого пара игнорируется\n"
    f"    L сейчас: <b>
{settings['min_volume']:,}$</b>\n\n"

    "⚙ <b>ПРОЧЕЕ</b>\n"

```

```

        f"/b BTC – добавить монету в чёрный
        список (без алертов)\n"
        f"    L в ЧС сейчас: <b>
{len(blacklist)} шт.</b>\n"
        f"/channel @имя_канала – куда
        дублировать сигналы (пусто = выкл)\n"
        f"    L сейчас: <b>
{settings['channel_id'] or 'Не задан'}
</b>\n"
        f"/s – показать текущий статус всех
        настроек одной сводкой"
        , parse_mode="HTML")

@dp.message(Command("channel"))
async def set_channel(message:
types.Message, command: CommandObject):
    settings["chat_id"] = message.chat.id
    if command.args:
        settings["channel_id"] =
command.args
        await message.answer(f"✅ Канал
        установлен: <b>{command.args}
        </b>\n<i>Сделай бота админом канала!</i>",
        parse_mode="HTML")
    else:
        settings["channel_id"] = None
        await message.answer("✅
        Дублирование в канал <b>ОТКЛЮЧЕНО</b>",
        parse_mode="HTML")

def _parse_float(args):
    return float(args.replace(',', '.'))

```

```

@dp.message(Command("p"))
async def set_percent(message:
types.Message, command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        settings["percent"] =
_parse_float(command.args)
        await message.answer(f"✅ Триггер
роста в окне: <b>{settings['percent']}%
</b>", parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /p 5")

```

```

@dp.message(Command("t"))
async def set_time(message: types.Message,
command: CommandObject):
    settings["chat_id"] = message.chat.id
    if command.args and
command.args.isdigit():
        settings["window_min"] =
int(command.args)
        await message.answer(f"✅ Окно: <b>
{command.args} мин</b>", parse_mode="HTML")
    else:
        await message.answer("❌ Ошибка.
Пример: /t 15")

```

```

@dp.message(Command("d"))

```

```

async def set_day_min(message:
types.Message, command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val = _parse_float(command.args)
        settings["day_min_rise"] = 0.0 if
val == 0 else abs(val)
        await message.answer(f"✅ Мин. рост
за 24ч: <b>{fmt_pct('day_min_rise')}</b>",
parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /d 5 (или /d 0 для выключения)")

```

```

@dp.message(Command("dmax"))
async def set_day_max(message:
types.Message, command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val =
abs(_parse_float(command.args))
        settings["day_max_rise"] = val
        await message.answer(f"✅ Потолок
роста за 24ч: <b>{fmt_pct('day_max_rise')}</b>",
parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /dmax 20 (0 = выключить)")

```

```

@dp.message(Command("wmin"))

```



```

async def set_week_min(message:
types.Message, command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val = _parse_float(command.args)
        settings["week_min_rise"] = 0.0 if
val == 0 else abs(val)
        await message.answer(f"✅ Мин. рост
за 7д: <b>{fmt_pct('week_min_rise')}</b>",
parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /wmin 10")

```

```

@dp.message(Command("w"))
async def set_week_max(message:
types.Message, command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val =
abs(_parse_float(command.args))
        settings["week_max_rise"] = val
        await message.answer(f"✅ Потолок
роста за 7д: <b>{fmt_pct('week_max_rise')}</b>",
parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /w 50 (0 = выключить)")

```

```

@dp.message(Command("mmin"))

```

```

async def set_month_min(message:
types.Message, command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val = _parse_float(command.args)
        settings["month_min_rise"] = 0.0 if
val == 0 else abs(val)
        await message.answer(f"✅ Мин. рост
за 30д: <b>{fmt_pct('month_min_rise')}
</b>", parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /mmin 20")

```

```

@dp.message(Command("m"))
async def set_month_max(message:
types.Message, command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val =
abs(_parse_float(command.args))
        settings["month_max_rise"] = val
        await message.answer(f"✅ Потолок
роста за 30д: <b>
{fmt_pct('month_max_rise')}</b>",
parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /m 100 (0 = выключить)")

```

```

@dp.message(Command("vh"))
async def set_vh(message: types.Message,
command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val =
abs(_parse_float(command.args))
        settings["vh_percent"] = val
        await message.answer(
            f"✅ Мин. превышение объёма
часа над средним по этому часу за 7д: <b>
{fmt_pct('vh_percent')}</b>",
            parse_mode="HTML")
    except Exception:
        await message.answer("❌ Ошибка.
Пример: /vh 200 (0 = выключить)")

```

```

@dp.message(Command("vm"))
async def set_vm(message: types.Message,
command: CommandObject):
    try:
        settings["chat_id"] =
message.chat.id
        val =
abs(_parse_float(command.args))
        settings["vm_percent"] = val
        await message.answer(
            f"✅ Мин. превышение объёма
минуты над средним за сутки: <b>
{fmt_pct('vm_percent')}</b>",
            parse_mode="HTML")
    except Exception:

```

```
await message.answer("❌ Ошибка.  
Пример: /vm 300 (0 = выключить)")
```

```
@dp.message(Command("v"))  
async def set_volume(message:  
types.Message, command: CommandObject):  
    settings["chat_id"] = message.chat.id  
    if command.args and  
command.args.isdigit():  
        settings["min_volume"] =  
int(command.args)  
        await message.answer(f"✅ Мин.  
объём 24ч: <b>  
{settings['min_volume']:,}$</b>",  
parse_mode="HTML")  
    else:  
        await message.answer("❌ Ошибка.  
Пример: /v 200000")
```

```
@dp.message(Command("b"))  
async def add_blacklist(message:  
types.Message, command: CommandObject):  
    settings["chat_id"] = message.chat.id  
    if command.args:  
        coin = command.args.upper()  
        pair = coin if  
coin.endswith("USDT") else f"{coin}USDT"  
        blacklist.add(pair)  
        await message.answer(f"🚫 <b>{pair}</b>  
</b> в ЧС", parse_mode="HTML")
```

```

@dp.message(Command("s"))
async def status_cmd(message:
types.Message):
    await message.answer(
        f"<b>Статус</b>\n"
        f"Триггер:
{settings['percent']}% за
{settings['window_min']} мин\n"
        f"24ч: мин
{fmt_pct('day_min_rise')} / потолок
{fmt_pct('day_max_rise')}\n"
        f"7д: мин
{fmt_pct('week_min_rise')} / потолок
{fmt_pct('week_max_rise')}\n"
        f"30д: мин
{fmt_pct('month_min_rise')} / потолок
{fmt_pct('month_max_rise')}\n"
        f"Объём/час vs неделя:
{fmt_pct('vh_percent')}\n"
        f"Объём/мин vs сутки:
{fmt_pct('vm_percent')}\n"
        f"Мин. объём 24ч:
{settings['min_volume']:,}$\n"
        f"Канал: {settings['channel_id']}
or 'Не задан'\n"
        f"В памяти алертов:
{len(alert_memory)}\n"
        f"Отслеживается пар:
{len(price_history)}"
        , parse_mode="HTML")

# ===== API =====

```

```

async def fetch_prices():
    url =
    "https://api.mexc.com/api/v3/ticker/24hr"
    try:
        async with aiohttp.ClientSession()
as session:
            async with session.get(url,
timeout=10) as response:
                if response.status == 200:
                    return await
response.json()
    except Exception as e:
        print(f"Ошибка API (ticker): {e}",
flush=True)
    return []

async def fetch_klines(symbol, interval,
limit):
    url =
    f"https://api.mexc.com/api/v3/klines?
symbol={symbol}&interval={interval}&limit=
{limit}"
    try:
        async with aiohttp.ClientSession()
as session:
            async with session.get(url,
timeout=8) as resp:
                if resp.status == 200:
                    return await
resp.json()
    except Exception:
        pass
    return []

```

```

async def get_long_term_changes(symbol,
current_price):
    """% изменения цены за 7д и 30д
(относительно закрытия свечи 7/30 дней
назад)."""
    data = await fetch_klines(symbol, "1d",
31)
    if not data:
        return 0.0, 0.0
    idx_7 = -8 if len(data) >= 8 else 0
    idx_30 = -31 if len(data) >= 31 else 0
    try:
        p_7 = float(data[idx_7][1])
        p_30 = float(data[idx_30][1])
        c_7 = ((current_price - p_7) / p_7)
* 100
        c_30 = ((current_price - p_30) /
p_30) * 100
        return c_7, c_30
    except Exception:
        return 0.0, 0.0

async def get_hour_volume_anomaly(symbol):
    """
    Берём последнюю ЗАКРЫТУЮ часовую свечу
(текущий формирующийся час не считаем,
т.к. его объём ещё не финальный) и
сравниваем с средним объёмом ЭТОГО ЖЕ часа
суток за предыдущие ~7 дней. Кэшируем
на BASELINE_TTL секунд на пару, чтобы не
заваливать API klines-запросами на

```

```

каждой итерации.
"""
    cached =
hour_baseline_cache.get(symbol)
    now = time.time()
    if cached and (now - cached["ts"]) <
BASELINE_TTL:
        return cached["cur"], cached["avg"]

    data = await fetch_klines(symbol, "1h",
192) # 8 дней с запасом
    if len(data) < 30:
        return 0.0, 0.0

    closed = data[:-1] # последний бар
    может быть ещё формирующимся
    last_closed = closed[-1]
    try:
        last_open_ts = int(last_closed[0])
        last_hour =
time.gmtime(last_open_ts / 1000).tm_hour
        last_vol = float(last_closed[7]) if
len(last_closed) > 7 else
float(last_closed[5])
    except Exception:
        return 0.0, 0.0

    same_hour_vols = []
    for bar in closed[:-1]:
        try:
            bar_ts = int(bar[0])
            bar_hour = time.gmtime(bar_ts /
1000).tm_hour
            if bar_hour == last_hour:

```



```

        vol = float(bar[7]) if
len(bar) > 7 else float(bar[5])
        same_hour_vols.append(vol)
    except Exception:
        continue

```

```

        avg = sum(same_hour_vols) /
len(same_hour_vols) if same_hour_vols else
0.0
        hour_baseline_cache[symbol] = {"ts":
now, "avg": avg, "cur": last_vol}
        return last_vol, avg

```

```

async def
get_minute_volume_anomaly(symbol):
    """
        Средний объём минутных свечей за
последние ~сутки (сколько отдаст API за
один
        запрос, обычно до 1000 баров  $\approx 16.64$  —
это приближение к "суточному" среднему,
        не строго 24ч, но по договорённости
используем это как базу для минуты вместо
        недельной статистики). Сравниваем с
последней закрытой минутой.
    """
        cached = day_baseline_cache.get(symbol)
        now = time.time()
        if cached and (now - cached["ts"]) <
BASELINE_TTL:
            return cached["cur"], cached["avg"]

        data = await fetch_klines(symbol, "1m",

```

```

1000)
    if len(data) < 30:
        return 0.0, 0.0

    closed = data[:-1]
    try:
        last_vol = float(closed[-1][7]) if
len(closed[-1]) > 7 else float(closed[-1]
[5])
        vols = [float(bar[7]) if len(bar) >
7 else float(bar[5]) for bar in
closed[:-1]]
    except Exception:
        return 0.0, 0.0

    avg = sum(vols) / len(vols) if vols
else 0.0
    day_baseline_cache[symbol] = {"ts":
now, "avg": avg, "cur": last_vol}
    return last_vol, avg

def pct_over_baseline(current, avg):
    if avg <= 0:
        return 0.0
    return ((current - avg) / avg) * 100

# ===== ОСНОВНОЙ ЦИКЛ
=====

async def parser_task():
    print("--- Фоновый парсер (pump/volume)
запущен ---", flush=True)

```

```

while True:
    try:
        data = await fetch_prices()
        now = time.time()
        window_sec =
settings["window_min"] * 60
        max_pts = max(int((window_sec /
settings["check_interval"]) * 1.2), 5)
        cooldown_sec =
settings["cooldown_min"] * 60

        for item in data:
            pair = item['symbol']
            if not
pair.endswith("USDT") or pair in blacklist:
                continue

            try:
                vol =
float(item['quoteVolume'])
                if vol <
settings["min_volume"]:
                    continue
                price =
float(item['lastPrice'])
                # MEXC отдаёт
priceChangePercent долей (0.0787 = 7.87%) –
умножаем на 100.
                ch_24 =
float(item['priceChangePercent']) * 100
            except Exception:
                continue

        if pair not in

```

```

price_history or price_history[pair].maxlen
!= max_pts:
    price_history[pair] =
deque(maxlen=max_pts)

    history =
price_history[pair]
    relevant = [p for (t, p) in
history if (now - t) <= window_sec]

    have_full_window = (
        len(history) > 0
        and (now - history[0]
[0]) >= window_sec
    )

    if have_full_window and
relevant:
        min_p = min(relevant)
        rise = ((price - min_p)
/ min_p) * 100 if min_p > 0 else 0.0
    else:
        min_p = price
        rise = 0.0

    if pair in alert_memory and
(now - alert_memory[pair]["time"]) >=
86400:
        del alert_memory[pair]

    # Базовая проверка цены
(24ч этаж/потолок – дешево, всегда доступна
из ticker)

    base_price_ok =

```

```

have_full_window and rise >=
settings["percent"] and ch_24 >=
settings["day_min_rise"]
        if settings["day_max_rise"]
> 0 and ch_24 > settings["day_max_rise"]:
            base_price_ok = False

        if not base_price_ok:
            history.append((now,
price))
            continue

        should_alert = True
        if pair in alert_memory and
(now - alert_memory[pair]["last_msg"]) <
cooldown_sec:
            should_alert = False

        if should_alert:
            ch_7, ch_30 = await
get_long_term_changes(pair, price)

            if
settings["week_min_rise"] > 0 and ch_7 <
settings["week_min_rise"]:
                should_alert =
False
                if should_alert and
settings["month_min_rise"] > 0 and ch_30 <
settings["month_min_rise"]:
                    should_alert =
False
                    if should_alert and
settings["week_max_rise"] > 0 and ch_7 >

```

```

settings["week_max_rise"]:
    should_alert =
False
        if should_alert and
settings["month_max_rise"] > 0 and ch_30 >
settings["month_max_rise"]:
    should_alert =
False

        vh_pct = vm_pct = None
        if should_alert and
settings["vh_percent"] > 0:
            cur_h, avg_h =
await get_hour_volume_anomaly(pair)
            vh_pct =
pct_over_baseline(cur_h, avg_h)
            if vh_pct <
settings["vh_percent"]:
                should_alert =
False

        if should_alert and
settings["vm_percent"] > 0:
            cur_m, avg_m =
await get_minute_volume_anomaly(pair)
            vm_pct =
pct_over_baseline(cur_m, avg_m)
            if vm_pct <
settings["vm_percent"]:
                should_alert =
False

        if should_alert:
            alert_memory[pair]

```

```

= {
    "time":
alert_memory[pair]["time"] if pair in
alert_memory else now,
    "price": price,
    "last_msg": now
}

base_coin =
pair.replace("USDT", "")
lines = [
    f"🚀 <b>ПАМП:
<code>{base_coin}</code></b>",
    f"📈 В окне
({settings['window_min']})м: <b>+
{rise:.2f}%</b>",
    f"📊 3а 24
часа: <b>{ch_24:.2f}%</b>",
    f"📅 3а 7 дней:
<b>{ch_7:.2f}%</b>",
    f"📅 3а 30
дней: <b>{ch_30:.2f}%</b>",
]
if vh_pct is not
None:
lines.append(f"📶 Объём/час vs неделя: <b>
{vh_pct:+.1f}%</b>")
if vm_pct is not
None:
lines.append(f"📶 Объём/мин vs сутки: <b>
{vm_pct:+.1f}%</b>")
lines.append(f"📶")

```

```

Было (мин. в окне): <code>{min_p}</code>")
        lines.append(f"💎")
Стало (тек): <code>{price}</code>")
        lines.append(f"💰")
Объём 24ч: <b>{int(vol):,}$</b>")
        alert_text =
"\n".join(lines)

        if
settings["chat_id"]:
            try:
                await
bot.send_message(settings["chat_id"],
alert_text, parse_mode="HTML")
            except
Exception as e:

print(f"Ошибка отправки админу: {e}",
flush=True)

        if
settings["channel_id"]:
            try:
                await
bot.send_message(settings["channel_id"],
alert_text, parse_mode="HTML")
            except
Exception as e:

                print(f"Не
удалось отправить в канал
{settings['channel_id']}: {e}", flush=True)

        history.append((now,
price))

```



```

        except Exception as e:
            print(f"Ошибка парсера: {e}",
                  flush=True)
            await
            asyncio.sleep(settings["check_interval"])

# ===== WEB & RUN
=====

async def handle_ping(request):
    return web.Response(text="OK",
                        status=200)

async def main():
    app = web.Application()
    app.router.add_get('/', handle_ping)
    runner = web.AppRunner(app)
    await runner.setup()
    site = web.TCPSite(runner, '0.0.0.0',
int(os.environ.get("PORT", 10000)))
    await site.start()

    await
    bot.delete_webhook(drop_pending_updates=True)
    asyncio.create_task(parser_task())
    await dp.start_polling(bot)

if __name__ == "__main__":
    asyncio.run(main())

```

